Project 3: Modular DBMS Engine

CmpE321 – Introduction to Database Systems

Spring 2026

Mert Ozustun, 2022400192

Selcen Celik, 2022400219

Department of Computer Engineering

Bogazici University

May 18, 2026

1 Introduction

This report describes our modular DBMS engine. It has four layers. From bottom to up they are the Disk Space Manager, the Buffer Manager, the File and Index Manager, and the Query Processor. Each layer is a Python package that interacts only with its lower layer. The entry point `archive.py` reads a JSON config file, builds the four layers, and feeds the input file line by line to the Query Processor.

The largest part of this report is Section 3. In that section, we describe every decision we made about sizing (integer width, string width, page header layout, record format, catalog entry, index pages) and provide reasons for all of our desicions. The instructor asked for this, so we made sure to keep it practical and provided calculations wherever needed. The subsequent sections explain each layer and then give the three experiments with real numbers from our own runs.

2 Architecture Overview

The spec says the four layers are wired bottom-up in `archive.py`:

```
disk     = DiskSpaceManager(config)
buffer   = BufferManager(config, disk)
file_idx = FileIndexManager(config, buffer)
qp       = QueryProcessor(config, file_idx, buffer, disk)
```

All calls between two layers produce a *Result object*, and not a bare value. The Result objects can be found in `models.py`:

- `PageResult` – raw page bytes and if a real disk I/O occured. Returned by the Disk Space Manager.
- `BufferResult` – a `PageResult` with cache-hit flag, evicted page id, and dirty-writeback flag. Returned by the Buffer Manager.
- `RecordResult` – the matching records plus pages accessed, index nodes visited, records scanned, the strategy used, and a status. Returned by the File and Index Manager.
- `WriteResult` – success flag, page id, and the bytes that were written. Returned by the Disk Space Manager on a write.

Since the data and the metadata always travel together within one of these objects, a layer can change its internals and the layer above does not have to change. For example, we can replace `LRU` with `MRU` or `heap_scan` with `bplus_tree` by editing single line of `config.json`. No code changes.

3 Sizing and Formating Decisions

All numbers below are based on a default `page_size` of 4096 bytes. The header size is independent of the page size, so that changing the page size only changes the body size.

3.1 Field byte widths

We record an integer field in **4 bytes**. We use Python `struct` with the format `"<i"`. This corresponds to a signed 32-bit little-endian integer. Range is about -2.1×10^9 to $+2.1 \times 10^9$. This is enough for normal data values and primary keys in this project. We didn't use 8 bytes because that doubles the integer cost for no benefit at the workload sizes the project asks for, and it would lower how many records fit on a page.

We keep a string field in **32 bytes**, null-padded. If the value is smaller than 32 bytes we pad the rest with `0x00`. If it is longer we truncate it to 32 bytes. We choose 32 since the spec says string values are alphanumeric only and the sample data (names like `Atreides`, `GiediPrime`) is short. 32 bytes fits those nicely and keeps the record small. Anyways, we want fixed width. Records need to be fixed length so we can find slot i by simple arithmetic instead of scanning.

Both choices are fixed per build, so decoding a record is just a walk over the field list adding 4 or 32 each step. No length prefixes are needed.

3.2 Page header layout

All pages (data, catalog, hash bucket, B+ tree node) have the same 32-byte header. The `struct` format is `"<BiI10si9s"`:

Offset	Size	Field
0	1	`page_type` (0 data, 1 B+ internal, 2 B+ leaf, 3 hash bucket, 4 catalog)
1	4	`page_id` (signed)
5	4	`record_count` (unsigned; key count for index pages)
9	10	`slot_bitmap` (one byte per slot, 0 free / 1 used)
19	4	`next_page` (chained pages; -1 means none)
23	9	reserved padding

We have intentionally made the header a fixed size for all page types. It means the body always starts at offset 32, so the slot math is the same everywhere and the code is still short.

The slot bitmap is **one byte per slot**, not one bit per slot. A bitmap with at most 10 slots is only 10 bytes either way, so bit-packing would save 9 bytes per 4096-byte page

(about 0.2%) and make the slot checks harder to read. The easier form of byte per slot was chosen. The 9 reserved bytes just pad the header out to a nice clean 32 so that future fields can be added without shifting the body.

3.3 Record format and page capacity

The body of a data page is $4096 - 32 = 4064$ bytes. Records are packed slot by slot beginning at offset 32. The size of a record is the sum of its field widths.

Using the sample example, the `house` type with 3 string and 3 integer fields:

$$recordsize = 3 \times 32 + 3 \times 4 = 108 bytes.$$

By raw space a page could hold $4064//108 = 37$ such records, but the config caps records per page at `max_records_per_page` (at most 10). Thus we only store 10 and the rest of the body stays empty. In this case, the limit is binding cap and not the byte space.

We examined the worst case as well. A type can have a maximum to 12 fields (see below). The biggest record is 12 string fields:

$$12 \times 32 = 384 bytes, \qquad 10 \times 384 = 3840 \leq 4064.$$

Thus, the largest possible type still can fit 10 records in a single page. TThis explains why at 4096 bytes this is a 10 byte bitmap and setting a max cap of 10 is always safe. The cap can never exceed the volume the page can hold.

3.4 System Catalog entry

The catalog is stored in `__catalog__.bin` in the same slotted-page format, so it survives restarts and every access goes through the Buffer Manager. One type definition per slot in the catalog.

We don't keep a parsed copy of the catalog in memory. Each metadata lookup requires a new scan of the catalog pages through the Buffer Manager. This keeps us close to the project spec and means an unexpected crash cannot leave a stale in-memory catalog behind; the only cache we rely on is the buffer pool itself.

- Type name: **16 bytes**. The spec requires at least 12; we used 16 to give a little headroom and to keep the entry aligned.
- Field name: **24 bytes**. The spec reuires at least 20; we used 24 for the same reason.

- Fixed header part: `"<16siiiiiii"` $= 16 + 7 \times 4 = 44$ bytes. The seven integers are number of fields, order of the primary key, number of data pages, number of records, B+ tree root page, number of hash buckets , and one reserved word.
- For each field descriptor: `"<24sB3s"` $= 24 + 1 + 3 = 28$ bytes (name, a one-byte type code, three padding bytes).

A type can contaion between 6 and 12 fields. The minimum is 6 (spec rule). We capped the upper bound at 12 so the catalog entry would be a fixed size no matter how many fields the type really has:

$$catalogentry = 44 + 12 \times 28 = 380 bytes.$$

With a body of 4064-byte that is $4064//380 = 10$ entries per catalog page, that fits with the 10-slot bitmap. When a catalog page is full we link a new one via the header's `next_page` field, so the number of types is unbounded.

3.5 Index page formats and fanout

Hash index (`<type>_idx.bin`). Pages $0..B-1$ are the primary buckets and are allocated when the type is created. We default to 16 buckets. One bucket entry is `"<32sii"` $= 40$ bytes: a 32-byte key (all keys serialized as a fixed 32-byte string so the bucket format is the same for int and str keys), a 4-byte data page id, and a 4-byte slot id. Each bucket page fits ten entries. When a bucket overflows we chain a new page through `next_page`. The hash function is the value mod the bucket count for integers, and the sum of char codes mod the bucket count for strings.

B+ tree (`<type>_idx.bin`). Each node is one page. The key is 4 bytes for an integer primary key and 32 bytes for a string primary key, which is determined when the type is created. With a 4064-byte body the raw fanout for an integer key is large (a leaf entry is key plus 8 bytes for the data pointer, so about $4064//12 \approx 338$). To limit the fanout, we impose a hard limit `HARD_FANOUT_CAP` of 50 keys per node. Then a node never needs more than one page, splits are cheap to write, and the tree remains shallow. The three stays two or three levels deep with a fanout of 50 for the data sizes for the project, so a point lookup requires only a few page reads. Delete is lazy: we delete the entry from the leaf but do not rebalance. The spec does not require rebalancing and a lazy delete is still correct, just less compacted.

3.6 Free space and page allocation

Free space is provided on two levels. The slot bitmap within a page indicates which slots are free. At the file level, each file keeps a small free list of page ids that were deallocated and can be reused. We prefer a per-file free list to a separate bitmap file because whole-page frees are rare in this workload (a normal delete just clears a slot, it does not drop the page), so a short list is sufficient and requires no extra file.

A page allocation is just bookkeeping. We increment the in-memory page count and return the new id without writing anything to disk. The real bytes are written later when the Buffer Manager flushes that page. Writing a zero page at allocation time would mean one more disk write that the buffer's later write-back makes pointless. A seek beyond the end of the file extends it automatically if the first real write lands.

3.7 I/O accounting decision

On a write we do *not* first read the old page. `WriteResult.old_data` exists for the Result-object contract but no caller reads it, and pre-reading would incur one fake disk read for every write. That will tie reads to writes and increase the read count. With our choice a write is exactly one write. This makes the experiment numbers honest and lets a write-heavy run actually show fewer reads than writes.

4 The Four Layers

4.1 Layer 1 – Disk Space Manager

This is the only layer that touches files. It reads and writes fixed-size pages with `seek()`; it never loads a whole file into memory. Each relation, each index, and the catalog is a separate `.bin` file beside `archive.py`. The path is derived from the script location, not from the working directory and not from the config, which is what the spec asks for.

It counts every read and every write and shows the totals. It also has a `log_write` stub that is called on every write. By default it does nothing, but it is there and is called, as needed. At startup it scans the folder for `.bin` files and recovers the number of pages in each file from the file size. This is how state survives a restart.

4.2 Layer 2 – Buffer Manager

The Buffer Manager is the page cache between Layer 3 and Layer 1. Layer 3 never calls the disk directly. The pool is an `OrderedDict` keyed by (file id, page id). An `OrderedDict`

provides O(1) move-to-end and O(1) removal from either end, which is exactly what the two policies need.

On a hit there is no disk I/O and we move the page to the end to mark it as recently used. On a miss we read the page from disk first (so a bad request cannot evict a good page), then make room if the pool is full. To make room we pick a victim by policy: `LRU` removes the page at the front of the order (least recently used), `MRU` removes the page at the back (most recently used). If the victim is dirty we write it back before dropping it. Writes are deferred: `write_page` just updates the cached copy and marks it dirty. The actual disk write happens on eviction or on the last `flush()`. We track requests, hits, misses, evictions, and dirty writebacks.

4.3 Layer 3 – File and Index Manager

This layer understands types, records, pages, and indexes. It reads the catalog through the buffer, coerces input tokens into the declared field types, and runs the operation with the current strategy.

- `heap_scan`: scan every data page and check each used slot. This is the baseline and it is also stored under the two indexes (the indexes only store (page, slot) pointers into the same heap).
- `hash_index`: a static hash on the primary key for equality lookups. The spec says heap scan is the fallback for a range query.
- `bplus_tree`: a B+ tree on the primary key to support equality and range lookups. A range query on a non-primary-key field reverts to a heap scan.

The index is created at the time of the type creation and it is kept updated on every insert and delete. Duplicate primary keys are rejected before anything is changed. All index pages are handled by the Buffer Manager.

4.4 Layer 4 – Query Processor

The Query Processor parses each line of input and passes it to Layer 3. It is responsible for the three output files. `output.txt` is cleared at the start of a run and contains query results. The `stats_output.txt` is overwritten on every `stats` command since it is a snapshot, not a log. The `log.csv` is append-only and is never cleared, so it survives restarts. All operations are logged as `timestamp,operation,status` where `int(time.time())` is used for the timestamp. Failures (duplicate key, missing type, range on a non-integer field, garbage input) are logged as `failure` and never cause the engine to crash.

Before creating the `stats` snapshot, the Query Processor calls `buffer.flush()`. First, pages that are still dirty in the pool are written , so they are counted as the disk writes they are about to become. This guarantees that the reported totals are for the entire workload, not just the I/O that happened to be on disk before the `stats` command was executed.

For `explain` we first query Layer 3 for an estimated strategy and I/O. Then we pull the type's catalog entry into the buffer *before* snapshotting the counters, so the catalog access that is always present is not counted as part of that one command. We snapshot the logical read and write request counters, run the command, and report the difference as "Actual I/O". Hit and missdeltas of the buffer show how the cache served those requests.

5 Configuration, Operations, and Persistence

In the config file, `page_size`, `max_records_per_page`, `buffer_pool_size`, `replacement_policy`, and `index_strategy` are set up. The engine is run as `python3 archive.py config.json input.txt` and only the config changes between test runs.

Supported operations are create type, create record, delete record (by primary key), search record (by primary key), range search (on any integer field), `explain`, `stats`, and `stats reset`. All data files, the catalog, the index files, and `log.csv` are persistent. If the engine is stopped and then started again on the same folder, it rebuilds its state from the `.bin` files and the catalog, and queries return the old data without requiring `create type`. We have directly verified this in our verification harness.

6 Experiments

All numbers below are from our own experiments. "I/O" is reads plus writes from the `Disk I/O` line. "Hit rate" is the percentage on the `Buffer Pool` line. The exact commands are in `record.txt` to reproduced the runs. The workloads are generated by `workload_generator.py` with `--seed 42`.

6.1 Experiment 1 – LRU vs. MRU

Setup: buffer pool size 8, index strategy `bplus_tree`. The sequential workload inserts 500 records and then does 50 full scans. The random workload inserts 500 records and then does 200 equality searches.

Table 1: LRU vs. MRU replacement policy comparison.

Workload	LRU I/Os	LRU Hit Rate	MRU I/Os	MRU Hit Rate
Sequential	37662	16.6%	35592	20.7%
Random	15972	20.9%	13686	31.7%

MRU was better than LRU in both lines. The sequential row is the sequential flooding problem. The full scan hits more pages than the pool can contain. With LRU, the scan finishes and the pool is filled with the latest pages.The next scan starts again at the first page and that is the page LRU just threw away. So scan after scan, almost every page is a miss. MRU discards the most recently used page and retains the older ones, so when the next scan begins some early pages are still in the pool and are hits. Therefore, MRU has the higher hit rate on repeated scans.

The random row is the same for a related reason. Every B+ tree search begins at the root and the upper internal nodes. Those get loaded early and, remain in the pool on MRU, while the leaf pages churning at the most-recent end and get evicted. Under LRU the root may drift towards the eviction end between searches and be dropped, so the next search has to read it again. MRU's higher hit rate here is due to retaining the few hot upper nodes.

6.2 Experiment 2 – Index Strategy Comparison

Setup: buffer pool size 16, replacement policy `LRU`. Equality works 300 records and 150 random equality searches. Range is based on 300 records and 150 random range queries on an integer field.

Table 2: I/O comparison across index strategies (reads + writes).

Query Type	HeapScan	HashIndex	B+-Tree
Equality	9121	5393	5148
Range	11795	9759 (heap fallback)	11019

As expected, the heap scan is the worst for equality, reading every data page until it finds the key.The hash index and the B+ tree go almost straight to the record, so do much less I/O. The B+ tree is slightly better than the hash index here since the hash buckets have some overflow chaining while the tree stays shallow.

The heap scan reads everything again for the range query. The spec requires the hash index to fall back to a heap scan for ranges, so it is doing the same kind of full scan, and its

lower number compared to plain heap is from a different buffer state left over from the insert phase (building the hash index touches different pages than building nothing), not from a better scan. The B+ tree does not save much here because the ranges generated are wide, often a large part of the key space. Still a lot visits most leaves and then follows pointers into most data pages plus the index pages on top. The B+ tree only really wins on selective ranges. It is still at or below the heap scan, which matches what we expect.

6.3 Experiment 3 – Buffer Pool Size Sensitivity

Setup: random workload (400 records, 200 equality searches), index strategy `bplus_tree`, replacement policy `LRU`. Only the buffer pool size is changed.

Table 3: Effect of buffer pool size on I/O performance.

Buffer Size	I/Os	Hit Rate
4	11023	23.5%
8	10717	25.2%
16	9481	33.1%
32	5079	63.8%
64	54	100.0%

The pattern is clean: more frames means less missed reads and less I/O. As the pool grows it can hold more of the working set (the upper B+ tree nodes and the hot data pages), so searches hit instead of miss. The interesting point is the jump at 64. The relation haas only 40 data pages,plus a small number of B+ tree pages. There are 400 records at 108 bytes and 10 records per page. The entire dataset fits in a pool of 64. After the initial warmup all requests are hits and the only disk I/O remaining is the 54 writes from the final flush of the dirty pages. Reads fall to zero and the hit rate is 100%. This also showing the write-back design working: nothing is written to disk until it has to be.

7 Conclusion

The engine matches the spec. The four layers are independent and only talk through Result objects, so the policy and the index strategy can be swapped from the config with no code change. We recorded each size we selected and showed the math that proves a page always contains its 10 records even for the largest allowed type. The three experiments behave as the theory predicts: MRU beats LRU under sequential flooding, the indexes beat the heap

scan for equality, and a larger buffer pool steadily lowers I/O until the data fits and disk reads disappear.